Quest—PM01 全身遥操作开发工作报告

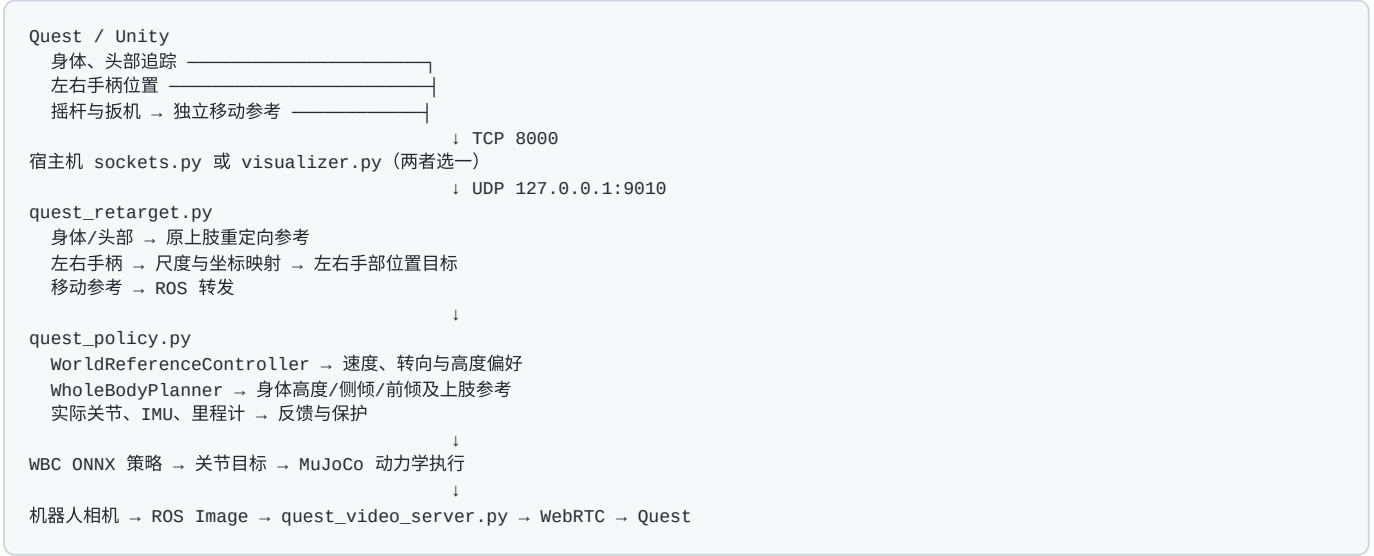
日期：2026-11-03。范围：本轮连续开发中涉及的策略控制、手柄移动、机器人视角、全身手部目标求解、数据转发和使用方法。本文按当前代码说明最终行为，测试结果区分自动测试、合成输入仿真与真人设备验证。

1. 工作目标与结果

原系统可以接收人体动作，但主要以原任务训练出的策略控制机器人，单纯改变上肢参考并不能让身体主动配合手部位置。此次工作将控制链路扩展为：用户用扳机控制行进和高度、摇杆控制机器人朝向，用左右手柄位置指定手部目标，由全身几何求解器给出身体姿态与上肢参考，再交给 WBC 策略执行。

原问题	当前实现	边界
不清楚哪些关节受到策略控制	明确区分原始 24 关节和 3 个仿真扩展关节	受到策略控制不等于每个关节都有独立用户目标
前后行进、下蹲与视角混淆	将移动参考与物理追踪分开，独立转发	不通过修改仿真位置实现移动
摇杆转的是视角	摇杆转向进入机器人偏航速度指令	头部方向仍来自物理追踪
前进方向像第三人称	前后移动按机器人实际基座朝向解释	最新方向修改需要重新部署 APK
手柄放低但身体不配合	左右手柄位置进入通用全身姿态求解	不只针对低位目标；仍有策略执行误差
机器人视角视频不显示	保留并检查 ROS 相机、WebRTC 和容器显示授权链路	网络地址、容器显示环境仍需正确
两种宿主机入口行为不一致	sockets.py 与 visualizer.py 均转发五类相关消息	两者不能同时占用 TCP 8000

2. 总体架构与数据流



`visualizer.py` 画出的是接收到的人体、手指和手柄追踪；它不代表机器人实际的行走轨迹，也不负责决定机器人的前进方向。全身求解和移动方向转换发生在机器人控制端。

3. 策略与关节控制关系

3.1 模式区别

启动模式	底座与腿部	上肢处理
不带模式参数	固定基座，上肢遥操作	原身体追踪重定向
`--walk` / `--run`	对应行走或跑步策略	对上肢实施参考覆盖，可能干扰策略平衡
`--wbc`	浮动基座，全身策略	上肢参考参与策略观测和残差动作，默认启用手柄全身求解

本次全身到达功能以 WBC 为主。默认模型为 `models/pm01_wbc/20260721_211647.onnx`。模型元数据决定关节顺序、观测缩放、动作缩放、PD 参数和允许范围，不把不同策略当作可直接互换的同一模型。

3.2 24 个策略关节与 27 个仿真关节

原始 24 个关节包含双腿 12 个、腰部 yaw 1 个、双臂 10 个和头部 yaw 1 个。策略输出覆盖原始 24 个关节。仿真额外增加头部 pitch 和左右手掌屈伸，共 27 个执行关节；这 3 个扩展量单独跟随追踪，不属于原始 ONNX 的 24 维输出，也不表示实机已具备对应能力。

WBC 的直接上肢参考覆盖双臂和头部 yaw 共 11 个原始关节。腰部和腿部没有各自独立的人体关节目标，但会受到身体姿态、移动和上肢参考的间接影响。腰部 yaw 仍由策略控制。原机器人没有独立腰部 pitch，因此“弯腰”需要由腿部与浮动基座姿态共同形成。

WBC 的关节目标可概括为：

```
q_target = action_offset + action_scale × policy_action + arm_delta
arm_delta = 限幅、限速后的上肢参考 − 默认上肢姿态
```

数组按模型动作关节顺序映射。`arm_delta` 也进入策略观测；策略保留学习到的修正，所以最终关节角不一定等于参考角。当前 WBC 输入为 872 维，输出为 24 维，控制频率为 50 Hz。最终由一个节点发布 27 关节命令，避免两个控制节点争抢同一执行接口。

4. 手柄按键、移动原理与坐标系

4.1 当前按键映射

输入	动作
任一摇杆横轴	机器人整体左转/右转；两侧同时输入时取绝对值较大的一侧
右食指扳机	沿机器人当前朝向前进
左食指扳机	沿机器人当前朝向后退
右握持扳机	提高身体高度偏好
左握持扳机	降低身体高度偏好
松开行进/转向输入	清除相应追赶目标，停止移动指令
松开高度输入	保存已设定的高度偏好

前后扳机、上下握持输入均作差，因此同时按下会相互抵消。摇杆纵轴不负责前后或升降。默认死区为 0.15，扳机越深，参考变化越快。Unity 的参考累计速度默认 1 m/s、竖直 0.6 m/s、转向 90°/s；这些是参考生成速度，不是实际机器人必然达到的速度。

4.2 为什么前进曾表现为第三人称

旧实现按 Quest 摄像机方向生成平面增量，并以启动时的朝向对齐到机器人世界。机器人后来转向时，旧世界目标没有同步跟随实际基座方向，因而可能表现为沿原方向走或侧移。

当前 `JoystickLocomotionProvider` 使用开始会话时冻结的方向编码有符号行进距离，不再让实时头部或摄像机转动改变行进方向。`WorldReferenceController` 将该编码解回局部增量，再用里程计测得的机器人实际 yaw 旋转到世界系：

```
Unity (right, up, forward) → 机器人平面 (forward, left)
局部增量 = R(初始操作方向) × 已转换的 Unity 增量
世界增量 = R(机器人当前 yaw) × 局部增量
```

机器人尚未完成的前向目标也随实际朝向重新对齐，丢弃横向残差。因此转向后继续按前进，目标会位于当前朝向的前方。使用的是机器人实际朝向，而非期望转角，避免机器人尚未转到位时就沿未来朝向横移。

4.3 世界参考如何变成真正行走

控制端维护短距离世界参考，再通过位置误差生成机器人局部速度：

```
世界位置误差 e = 目标位置 − 实际位置
局部平面速度 = R(−实际 yaw) × 1.5e
偏航速度 = 1.5 × 限幅后的偏航误差
```

平面目标超前距离限制为 0.25 m，相应速度模长上界为 0.375 m/s，随后还受模型自身范围限制。偏航误差限制为 ±0.8 rad，可产生最大 ±1.2 rad/s 的请求。旧的较低转向限制曾出现转向不充分，因此恢复使用模型允许的转向范围。

松开输入时，将目标重置到当前位置或当前 yaw，不继续追赶旧目标；反向输入会先清除未完成的正向目标。会话编号和递增序号过滤过期、重复移动数据；超时 300 ms 停止移动，重连重新锚定，避免把断线期间累计偏移一次性执行。

机器人通过策略产生的腿部动作改变实际位置。代码没有直接写入真实仿真基座位置来实现平移或转向。“松开即停止”指停止速度请求，物理系统仍可能短暂调整姿态或存在惯性。

5. 从手柄位置到全身姿态

5.1 输入来源与位移去重

按用户选择，位置目标采用左右控制器位置。头部和手腕朝向继续使用原追踪。Unity 仅在控制器位置追踪有效时发送对应包。

控制器包附带 `| locomotion = x y z yaw` 元数据，机器人端先移除已应用于追踪帧的虚拟偏移，避免按键移动又被解释为人的手部运动。当前发送端追踪姿态保持物理方向，元数据中的旋转为零；解析器保留对旋转补偿的支持。

旧 APK 不带元数据的控制器包不会用于新的手部位置求解，以免双重计算移动。因此更新 Python 代码不能替代重新部署 APK。

5.2 HandTargetMapper：位置映射

每次 Streaming 会话首次有效身体帧确定人体方向基、初始肩部锚点和臂长比例，此后保持这些基准不变。每侧手目标为：

```
p_target = 机器人肩部锚点
         + 臂长比例 × 初始人体方向变换 × (当前控制器位置 − 初始人体肩部位置)
```

这保留了手部距离和身体移动的影响，不把手臂向量归一化，也不在每帧随人体弯腰重新转动目标坐标。否则人体弯腰带来的位移可能被“以当前身体为参考”的变换抵消。

目标采用机器人朝向坐标，轴为前、左、上；机器人肩部锚点按默认站立高度调整。手部水平目标随机器人整体移动/转向，不是外部世界固定物体的坐标。人的地面与机器人地面没有经过精确配准，手柄贴地不意味着机器人手端能精确贴地。

使用时间常数 0.08 s 的一阶滤波平滑目标。个人 `--body-profile` 提供人体臂长、增益和机器人手端配置；新位置映射不等同于旧的逐段手臂方向重定向，也不复用全部旧肩部偏移标定。

5.3 WholeBodyPlanner：优化变量与约束

求解状态包含 23 个变量：基座高度、roll、pitch，加双腿 12 个关节及双臂 8 个肩/肘俯仰关节。基座水平位置和 yaw 固定在当前机器人朝向坐标中；腰部 yaw 保留反馈值。头部、双侧 elbow_yaw 和新增手掌

屈伸由原追踪参考提供。

优化同时考虑：

残差	在残差与雅可比上的乘数	意义
双手位置误差	8	尽量让手端到达目标
双脚位置误差	20	尽量保持当前支撑位置
双脚旋转误差	3	减少脚部不合理转动
身体高度偏好	0.65	在手部目标允许时靠近期望高度
身体 roll/pitch 偏好	0.35	倾向恢复直立
关节参考偏好	0.035	约束冗余姿态变化

这些数值是残差缩放系数，进入平方损失后对应平方权重。约束是软约束，不能理解为脚被绝对固定。每次求解使用实际反馈重建脚部锚点，避免手动行走时还追赶历史脚位。

使用 MuJoCo 解析雅可比和阻尼最小二乘：

```
Δx = (JᵀJ + 0.025I)⁻¹Jᵀe
```

每次最多 14 次迭代，使用上次结果热启动。单次迭代的高度步长限 0.04 m、姿态角步长限 0.08 rad、关节步长限 0.12 rad；同时裁剪模型关节范围与策略上肢范围。求解约每三个控制周期进行一次，即约 16.7 Hz，策略仍保持 50 Hz。

求解器仅操作独立的 `MjData`，不会直接改写仿真机器人状态。几何解中的腿部角度用于确定可行身体姿态，并未直接覆盖策略腿部命令；实际执行仍依赖 WBC。

5.4 姿态参考进入 WBC

移动与姿态指令排列为：

```
loco_command = [vx, vy, yaw_rate, height, yaw, pitch, roll]
```

独立身体 yaw 槽保持零；转身使用 yaw_rate。全身求解器给出的 `[height, roll, pitch]` 会正确映射到对应槽位，上肢部分进入原有参考通道。

为补偿观察到的高度跟踪偏差，高度命令采用：

```
h_command = clip(h_plan + 1.5 × (h_plan − h_actual), 模型高度范围)
```

随后高度变化率限制为 0.2 m/s，身体倾角参考变化率为 0.5 rad/s，上肢参考变化率为 1.5 rad/s。当前规划前倾上限为 0.75 rad，后倾下限为 −0.3 rad，roll 为 ±0.3 rad，并与模型范围取交集。倾倒超过 0.9 rad 的原保护保留。

握持扳机保存的是高度偏好；开启全身求解后，为够到手部目标，求解器可能选择不同高度。因此“保存下蹲高度”不等于强制锁死最终身体高度。若需要只使用原上肢参考及手动高度，可加 `--upper-only`。

6. 转发、视频及关键文件

6.1 两种接收入口

`sockets.py` 和 `visualizer.py` 均转发：`Body joints_pose`、`Head pose`、`Left controller_wrist`、`Right controller_wrist`、`Locomotion reference`。元数据原样保留。TCP 按换行重组，支持拆包和粘包；不把断开连接时的残缺末行当作完整数据。

`sockets.py --tally` 仅改变终端显示方式，不禁用转发。visualizer 的人体/手指绘制与控制转发是两项独立功能。最新的机器人朝向移动逻辑无需在可视化绘图端重复实现。

6.2 机器人视角视频

视频链路为机器人相机 ROS 图像，经 `quest_video_server.py` 转为 WebRTC 流，Quest 接收显示；默认信令端口为 8765。TCP 8000 是追踪数据通道，不是机器人视频端口。

此前工作处理了接收地址和容器显示授权，并验证过 1280×720 图像。`authorize_quest_display.sh` 用于显示授权，启动入口会在适用时读取 `/tmp/quest-display.Xauthority`。保存的局域网地址不是永久地址，换网络后仍需核对。单独运行 sockets 或 visualizer 不会启动机器人相机；应启动完整 WBC 入口。

6.3 实现文件索引

以下路径以仓库根目录为起点：

文件	主要职责
`hand-tracking-streamer/hand_tracking_streamer/Assets/Scripts/JoystickLocomotionProvider.cs`	按键映射、会话与序号、移动增量、追踪偏移元数据
`hand-tracking-streamer/hand_tracking_streamer/Assets/Scripts/ControllerPalmPoseStreamer.cs`	控制器追踪有效性与手柄位置数据发送
`hand-tracking-streamer/scripts/sockets.py`	轻量 TCP/UDP 接收与转发
`hand-tracking-streamer/scripts/visualizer.py`	人体/手指显示及相同控制数据转发
`wholebody_teleop_ws/scripts/prepare_quest_teleop.py`	生成运行场景、模型配置、默认高度
`wholebody_teleop_ws/scripts/quest_retarget.py`	原身体重定向、控制器目标映射、ROS 发布
`wholebody_teleop_ws/scripts/quest_locomotion.py`	元数据补偿、世界参考、当前朝向移动、停止与重连
`wholebody_teleop_ws/scripts/quest_wholebody.py`	手部目标映射、全身几何求解、高度补偿
`wholebody_teleop_ws/scripts/quest_policy.py`	观测构造、策略推理、姿态融合、反馈保护、关节命令
`wholebody_teleop_ws/scripts/quest_video_server.py`	ROS 图像至 WebRTC 视频
`wholebody_teleop_ws/scripts/run_quest_teleop.sh`	统一启动、参数分发、子进程退出清理
`wholebody_teleop_ws/scripts/tests/test_quest_locomotion.py`	行进、转向、停止、反向、朝向及数据时序测试

文件	主要职责
`wholebody_teleop_ws/scripts/tests/test_quest_wholebody.py`	全身到达、恢复、雅可比、范围、映射测试

7. 完整操作方法

7.1 更新发送端

在 Unity 打开 `hand-tracking-streamer/hand_tracking_streamer`，将最新脚本重新 Build And Run 到 Quest。此次控制器元数据、有效追踪判断和前进方向编码均涉及 APK。C# 编译通过不等于 APK 已构建或安装，本轮未自动完成设备部署。

7.2 宿主机接收数据

在宿主机终端：

```
conda activate hts
cd /media/youran/3C50A006509FC554/Users/12911/Documents/Study/TUM/Masterarbeit/Learning_Based_Control/hand-tracking-streamer
```

需要可视化时：

```
HTS_SIM_FORWARD_PORT=9010 python scripts/visualizer.py --protocol tcp --host 0.0.0.0 --port 8000 --show-fingers
```

只接收并显示统计时：

```
HTS_SIM_FORWARD_PORT=9010 python scripts/sockets.py --protocol tcp --host 0.0.0.0 --port 8000 --tally
```

两条命令二选一。Quest 数据连接指向宿主机当前局域网 IP 和端口 8000；`0.0.0.0` 是宿主机监听地址，不是填写给 Quest 的目标地址。

7.3 启动机器人端

在开发容器内：

```
cd /workspaces/Learning_Based_Control/wholebody_teleop_ws
bash scripts/run_quest_teleop.sh --wbc
```

使用已有个人标定：

```
bash scripts/run_quest_teleop.sh --wbc --body-profile configs/quest_calibrated.yaml
```

对照原上肢模式：

```
bash scripts/run_quest_teleop.sh --wbc --upper-only
```

入口负责启动模拟器、重定向、策略和视频服务。默认 ROS_DOMAIN_ID 为 87，ROS_LOCALHOST_ONLY 为 1。请停止上一次实例再启动；Ctrl+C 会清理该入口创建的子进程和临时配置。这些 Python/C# 改动不要求重新编译 ROS，但需要已有可用的工作空间构建和依赖。

7.4 推荐操作顺序

1. 启动一种宿主机转发程序，再启动容器 WBC 入口。
2. 确认 Quest 连接地址与视频地址对应当前电脑网络。
3. 自然站直、面向前方，保持身体和双手柄追踪有效，再开始 Streaming。
4. 等待身体追踪、两侧手柄目标和机器人反馈接入。
5. 小幅前伸、降低、侧移手柄，观察双臂和身体高度/姿态是否共同响应。
6. 用摇杆转动机器人，再按右食指扳机，确认沿机器人新的实际朝向前进；左食指扳机后退。
7. 松开检查停止请求；握持扳机调整高度偏好，观察其与手部到达目标的共同作用。

重新开始 Streaming 会重建人体坐标基准。若以已经弯腰或明显扭转的姿态开始，会影响本次位置映射。

8. 诊断、丢失追踪与常见问题

现象	优先检查
只有手臂动、身体不配合	是否用 `--wbc`；是否误加 `--upper-only`；APK 是否含控制器元数据；是否收齐两侧目标
看得到人体但机器人不动	visualizer 是否设置转发端口 9010；容器重定向是否接入；身体数据是否持续有效
转身后前进仍沿旧方向	APK 和机器人 Python 是否都已更新并重启；不能只更新转发端
手柄很低但手端没到地面	映射比例/初始姿态、机器人可达范围、姿态限制与策略执行误差
松开高度后身体仍变化	高度是偏好，全身求解仍会为手部目标调整高度
视频不显示但姿态数据正常	模拟器是否发布新图像、视频服务是否运行、8765 地址与网络、容器显示授权
地址已被占用	sockets 和 visualizer 是否同时运行，或是否有旧实例
机器人进入被动阻尼	实际反馈超时或倾角超过保护阈值；需重置仿真

手柄一侧失去追踪时保持该侧最后目标，另一侧继续更新；两侧都失去追踪时停止重新求解并保持最后姿态参考。这里的“保持”是保持参考，策略仍持续执行平衡控制。身体数据超过 300 ms 无新帧时停止移动请求；机器人关节或 IMU 反馈超过 300 ms 时进入被动阻尼。

可观察的诊断接口包括：

- `/simulation/quest/left_hand_target`、`/simulation/quest/right_hand_target`：`PointStamped`，frame_id 为 `quest_robot_heading`。
- `/simulation/quest/wholebody_reference`：高度、roll、pitch 与上肢参考；其中高度是几何目标，不是经过反馈补偿后的最终命令。
- `/simulation/quest/upper_reference`：原追踪重定向参考。
- 日志 `Whole-body reach enabled`：功能已启用，等待控制器目标。
- 日志 `Whole-body hand reach`：几何规划误差与实际反馈误差，单位厘米。

9. 验证结果及证据范围

9.1 自动测试与编译

全身功能阶段的 59 项测试通过。其中批量运行首次漏加载 ROS 环境，视频模块导入失败；补加载 ROS 后单独运行 4 项视频测试通过，其余 55 项已通过。后续当前朝向移动修改后，14 项移动控制测试通过，包含新增的转向后前进、保留目标方向重对齐、非零人体初始朝向和反向检查。这里不是宣称最终修改后重跑了完整 61 项套件。

Unity C# 脚本编译通过，有既存的文本换行 API 过时和无 await 的 async 警告。没有据此宣称完成 Android APK 部署。shell 语法与差异格式检查通过。

9.2 通信链路测试

合成数据完整经过 visualizer → UDP → retarget → 全身 IK → WBC 命令，收到了 73 条姿态参考和 218 条关节命令。停止左手数据后左目标计数停止增长，右侧继续更新。该测试使用合成机器人反馈，用于验证连接与消息行为，不能证明动态到达精度。

另对 sockets 的 TCP tally 模式验证了五类消息原样转发，包含拆分输入重组。新移动方向逻辑由移动控制测试覆盖，尚未在真实 Quest 操作中完成复验。

9.3 几何与动力学验证

几何求解验证了前伸、低位、侧移、高位、目标返回、不可达目标限幅及雅可比。解析雅可比与有限差分最大误差约 2.2×10^{-7}；本地单次求解约 5–9 ms，属于该测试环境测量，不能视为实时性保证。

16 秒 MuJoCo + WBC 动态测试：双手目标前移 20 cm、降低 18 cm，再返回。记录如下：

阶段	实际基座高度	实际 pitch	实际左右手误差
初始稳定附近	0.737 m	约 2°	—
低位保持后期	0.622 m	约 16.7°	约 8.3 / 6.7 cm
返回后	0.730 m	约 3.8°	约 4.3 / 3.0 cm

测试未触发 0.9 rad 倾倒保护。高度反馈补偿改善了该案例：未补偿时低位实际手部误差约 14–15 cm。该结果只支持“当前案例下全身参与并改善到达”，不支持所有目标均能精确完成。

10. 当前能力边界与后续工作

系统目前实现的是“位置目标 → 全身几何参考 → 学习策略执行”，并非重新训练后的端到端精确手部跟踪策略。几何解误差很小、实际误差较大时，优先分析策略跟踪能力和反馈，而不是直接认为背部关节被锁住。

目前没有新增障碍物避碰、显式质心/支撑多边形约束或为够到目标自动规划迈步。脚部约束是软约束，运动时锚点随反馈更新；复杂步态、强烈弯腰和大范围伸手的组合尚不能据现有测试认定稳定。规划范围、关节限位、人体尺度和模型训练分布都会影响结果。

后续优先完成新版 APK 的真人验证，记录手部目标、几何参考、实际关节与实际手端位置，对前伸、侧移、下蹲和转向行进分别量化误差。若策略持续抵消合理参考，再考虑调整参考接口、代价权重或针对任务训练策略。应先通过日志区分映射误差、几何不可达和策略执行偏差，避免只扩大姿态范围掩盖问题。